Introduction to LAMBDAS

As we know that java's whole principle works on **Object Oriented language**, everything in java revolves around **Object**.

Que→ Why do we need behaviour passing in functions ?

Ans→ taki hmara code lengthy na bane or easy to use rhe.

Comparator Interface

- **It has only one method** → compare(o1, o2);

Comparator Interface with creating object of it

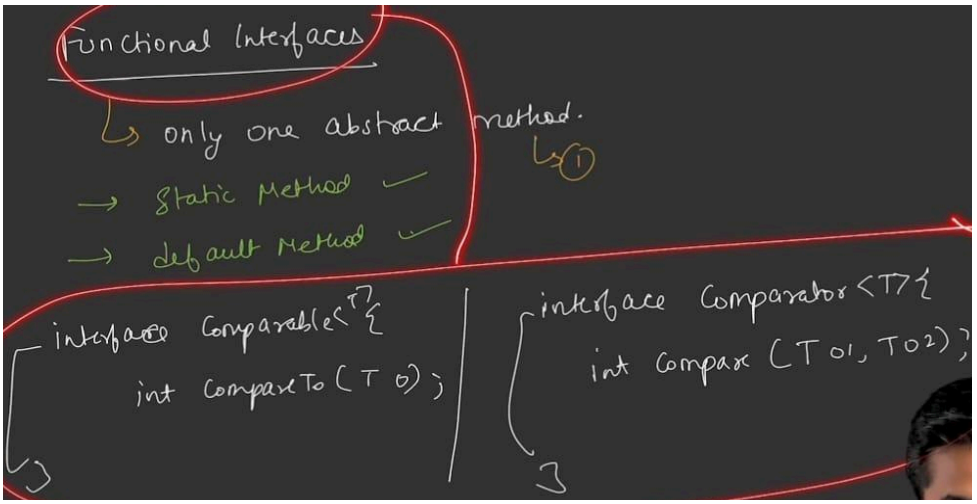
```
import java.util.*;

public class ComparatorInterfaceeee {
    public static void main(String[] args) {
        List <Student> list = new ArrayList<>();
            list.add(new Student("Aryan", 01, 93));
            list.add(new Student("Priyansh", 02, 96));
            list.add(new Student("Tanuja", 03, 83));
            list.add(new Student("Raghvindra", 04, 94));
                Comparator<Student> c1 = new SortByName();
                Comparator<Student> c2 = new SortByRollNo();
                Comparator<Student> c3 = new SortByMarks();
            Collections.sort(list, c2);
                for(Student s : list){
                    System.out.println(s.name + ", " + s.rollNo + ", " + s.marks);
                }

    }
}
class SortByName implements Comparator<Student>{
    @Override
    public int compare(Student s1, Student s2){
        return s1.name.compareTo(s2.name);
    }
}
class SortByRollNo implements Comparator<Student>{
    @Override
    public int compare(Student s1, Student s2){
        return s1.rollNo - s2.rollNo;
    }
}
class SortByMarks implements Comparator<Student>{
    @Override
    public int compare(Student s1, Student s2){
        return s1.marks - s2.marks;
    }
}
class Student{
    String name;
    int rollNo;
    int marks;
        public Student(String name, int rollNo, int marks){
            this.name = name;
            this.rollNo = rollNo;
            this.marks = marks;
        }
}
```

Hence above code is very lengthy to perform sort action, so to solve this problem java gives us the **ANONYMOUS class**, but still the code is **verbose** and lengthy hence comparator Interface is still wrappered inside the Anonymous class.

So, further then the problem was solved by java by introducing the **FUNCTIONAL INTERFACE**.
Functional Interface → The Interface with only one **ABSTRACT method** declared in it.



Hence, **Comparable Interface & Comparator Interface** both are the **functional Interfaces.**
Only because of Functional Interface it is possible to usee LAMBDA EXPRESSION, because it has only one abstract method in a particular Interface, too which LAMBDA can Map too->.

LAMBDA Expression

Lambda Expression→ (Parameters) -> expression, and **(->)** symbol means **Maps to / Gives to.**
Therefore Lambda expression me hum parameters me class ke parameters lete ha or or expression me functional interface ka abstract method lenge.
The code bellow is the modified code of the above comparator interfaces code, and here the lengthy code is replaced by the LAMBDA expression..

```
 public static void main(String[] args) {
        List <Student> list = new ArrayList<>();
            list.add(new Student("Aryan", 01, 93));
            list.add(new Student("Priyansh", 02, 96));
            list.add(new Student("Tanuja", 03, 83));
            list.add(new Student("Raghvindra", 04, 94));
               Collections.sort(list, (s1, s2) -> s1.name.compareTo(s2.name));
               for(Student s : list ){
                System.out.println(s.name + ", " + s.rollNo + ", " + s.marks);
               }
    }
}
class Student{
    String name;
    int rollNo;
    int marks;
        public Student(String name, int rollNo, int marks){
            this.name = name;
            this.rollNo = rollNo;
            this.marks = marks;
        }
}
```

How does in the Lambda Expression argument it knows that s1 & s2 are the parameters of the Student class ?
Because it knows that the comparator interface is implemented by the Student class bellow, from the current context & this type of approach is known as **TARGET TYPING.**

Some different ways to declare LAMBDA EXPRESSION →

- **MULTIPLE PARAMETERS:** (a, b) -> a+b
- **SINGLE PARAMETER:** (x) -> x * x
- **NO-PARAMETER:** () -> System.out.println(“ Hello”);
- **MULTLINE LAMBDA:** (a, b) -> {int sum = a+b;

return sum;}

Hum apna khud ka functional Interface bhi bna skte ha,

```
@ functional Interface // this is optional just to get the compile time safety, taki isme koi or ABSTRACT method add na krr ske..
interface Demo {
    m1();
}
```